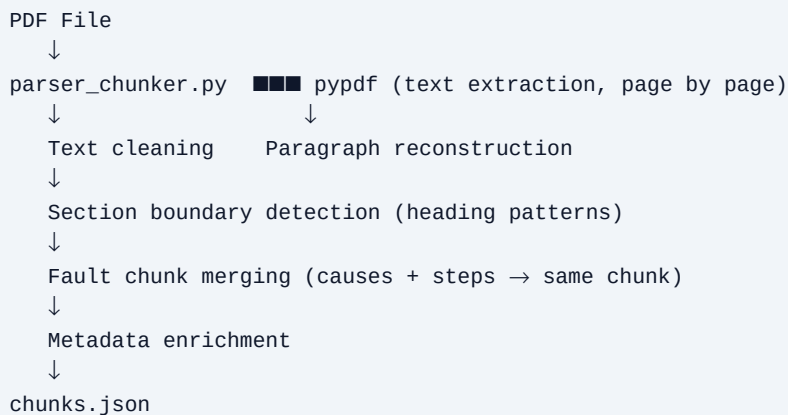


07 — Chunking & Parsing Strategy

How hardware manuals are converted into self-contained, retrieval-friendly chunks for the RAG pipeline.

Overview

The chunking pipeline converts raw PDF manuals into structured chunks that feed the vector search and LLM completion steps. Each chunk is semantically self-contained — typically one error code, one maintenance task, or one specification section — with metadata tags for filtering and cross-referencing.



Implementation: Single file at `src/ingestion/parser_chunker.py`. Exposes `process_and_chunk()` and `save_chunks_to_json()`. Also mounts a FastAPI endpoint at `POST /parse-manual`.

Important Note on Document Scope

Note: The parser and chunker are validated against five test manuals provided during development, not the full range of production documentation. Format detection patterns, heading recognition, and error code regex are tuned to these five documents. They will need review and extension for new production manuals.

Test Document Formats

Document	Error Code Format	Causes Label	Resolution Label
LC-2040 Laser Cutter	E-01 ... E-08	Common causes:	Resolution Steps:
IMM-750 Injection Moulder	E-01 ... E-08	Common causes (low/high):	Resolution Steps:
FDM-X300 3D Printer	ERR-01 ... ERR-07	Why it happens:	How to fix it:
RA-6200 Robot Arm	F-01 ... F-07	Common causes:	Diagnostic steps and resolution:
HP-500 Hydraulic Press	A-01 ... A-07	CAUSES:	RESOLUTION:

Evolution of the Approach

■ Attempt 1 — Docling + MarkdownHeaderTextSplitter

Approach

Docling for PDF → markdown conversion. LangChain's `MarkdownHeaderTextSplitter` to split on `##` boundaries.

Why it failed

- Each error code fragmented into 3–5 pieces (description, causes, resolution chunks). No single chunk contained enough context.
- Parenthetical sub-heading variants (e.g., *Common causes (low pressure):* vs *Common causes (high pressure):*) never merged back.
- Docling's element tree API changed between versions; code accessing `.label.name` and `.prov` failed silently on some pages.
- `do_table_structure=True` caused `std::bad_alloc` on dense pages, silently dropping content.

■ Attempt 2 — Docling + Custom Merge (Regex Prefix)

Approach

Keep Docling. Replace header splitter with a custom pass: detect sub-heading prefixes and fold them into parent error-code chunks.

Why it failed

- Required manually maintained vocabulary of sub-heading prefixes. New documents with different phrasing broke the merge.
- FDM-X300 uses conditional branches as sub-headings (*If temperature rose above set point:*). No prefix list could enumerate all conditionals.
- Docling still failed on dense pages; the markdown normaliser that was added to fix PyMuPDF fallback became itself another fragile pattern list.

■■ Attempt 3 — pypdf + LLM Segmentation (Groq / Llama)

Approach

Drop Docling. Use `pypdf` for extraction. Send page batches to Llama 3.3 70B (Groq) with 1-page overlap, asking the LLM to identify boundaries and extract metadata.

Partial failures

- Groq's free tier TPM limit (12,000) was exceeded with full 9-page manuals. API returned HTTP 413.
- 2-page batches couldn't see error code sections spanning 2–3 pages. Causes/resolution steps on the next page appeared as orphaned fragments.
- Hash-based deduplication removed the longer (complete) version of a chunk when the shorter version appeared first.

Partial fixes applied

- Adaptive batch sizing (small docs → single call; large → smaller batches).
- Smarter deduplication: keep the longest version of each error code across all batches.
- `temperature=0` for deterministic JSON output.
- Explicit prompt: don't return an error code if its resolution isn't visible in the current batch.

■ Attempt 4 — pypdf + Rule-Based Segmentation (Final)

Approach

Keep pypdf for extraction. Replace LLM segmentation with deterministic rule-based approach.
Document-format-aware heading patterns + step-merging pass.

Why this was chosen

- **No API dependency**, no token limits, no rate limiting.
- **Deterministic**: same input always produces same output.
- **Fast**: 10-page manual processes in < 1 second.
- **Explainable**: every chunk boundary can be traced to a specific pattern.
- **Sufficient** for the test documents. Patterns are documented and extensible.

■ Implementation Details

Text Extraction

Library: pypdf (via PdfReader). pypdf extracts text page-by-page as a raw string. For digitally-generated PDFs (not scanned), it recovers all text without OCR.

Known issue: pypdf sometimes emits body text one word per line. A paragraph reconstruction step fixes this.

Paragraph Reconstruction

The `_reconstruct_paragraphs()` function collapses word-per-line fragments into full logical lines. **Standalone lines** (headings, steps, bullet points, sentence-ending lines) are kept as-is. **Short continuation fragments** are accumulated in a buffer and flushed when a standalone line is encountered.

The `_is_standalone()` classifier uses these heuristics (in order):

- Fault heading pattern match → always standalone
- Section heading pattern match (≤ 8 words) → standalone
- Maintenance frequency heading match
- Sub-heading match (Common causes:, Resolution Steps:, etc.)
- Numbered step match (1. ..., 2) ...)
- Bullet prefix match (●, —, -)
- ALL-CAPS heading match (HP-500 style)
- Sentence-ending check (ends with .!? and has > 3 words)
- Short fragment (≤ 2 words) → continuation, not standalone

Section Boundary Detection

Headings are classified into four types by `_heading_type()`:

Type	Patterns	Examples
fault	Error/Fault/Alarm code patterns	Error Code E-01 —, Fault Code F-03 —, ALARM A-04 —, ERR-07 —
section	Numbered section headings ≤ 8 words	1. Machine Overview, Section 2 — Technical Specifications
maintenance	Frequency keywords at line start	Daily, Weekly, Monthly, Every 6 Months, After Every Print
none	Everything else	Body text, steps, bullets, sub-headings

Fault Chunk Merging

Core design decision: Each error code becomes one self-contained chunk. When `_split_into_sections()` encounters a `fault` heading, it opens a new chunk. Every subsequent line — including numbered steps, bullets, and sub-headings — stays in that chunk until the next `fault`, `section`, or `maintenance` heading.

Result:

- `Common causes`: + bullets → stay in error chunk
- `Resolution Steps`: + numbered steps → stay in error chunk
- Conditional branches (*If temperature rose above:*) → stay in error chunk
- Sub-headings don't trigger chunk boundaries; they only affect line formatting

Chunk Parameters

Parameter	Value	Rationale
<code>MIN_CHUNK_CHARS</code>	60	Drop bare-header sections with no body
Error code chunk size	~600–1800 chars	Natural — complete error code with description, causes, all steps
Maintenance chunk size	~200–600 chars	Natural — one frequency group (Daily, Weekly, etc.)
Specification chunk size	~400–1200 chars	Natural — one spec table or parameter section
Overlap	None	Segmentation is deterministic. Overlapping chunks would duplicate without adding value.

Metadata Fields

Every chunk carries the following metadata for filtering and analysis:

Field	Type	Description
<code>machine_id</code>	<code>str</code>	Normalised document ID (spaces/hyphens → underscores, uppercased)
<code>source_file</code>	<code>str</code>	Original PDF filename
<code>source</code>	<code>str</code>	Original PDF filename (duplicate for API compatibility)
<code>section</code>	<code>str</code>	Raw heading text of this section
<code>section_id</code>	<code>str</code>	Same as <code>section</code> — label for retrieval filtering
<code>page_numbers</code>	<code>list[int]</code>	All physical page numbers this chunk spans
<code>page_start</code>	<code>int</code>	First physical page (1-based)
<code>page_number</code>	<code>int</code>	Last physical page (1-based)
<code>is_table</code>	<code>bool</code>	True if content contains markdown table syntax
<code>is_safety</code>	<code>bool</code>	True if content matches safety keywords
<code>is_troubleshooting</code>	<code>bool</code>	True if <code>chunk_type == error_code</code> or troubleshooting keywords present
<code>chunk_type</code>	<code>str</code>	Enum: <code>error_code</code> , <code>maintenance</code> , <code>specification</code> , <code>safety</code> , <code>spare_parts</code> , <code>overview</code>

Field	Type	Description
error_code	str null	Bare error code (E-01, F-03, A-04, ERR-07) or null
cross_references	list[str]	Other error codes cited in this chunk's content

Coverage Validation

After all chunks are produced, `_validate_coverage()` scans the full PDF text for all error code patterns and cross-checks them against the chunk set.

Output: Any error code present in the PDF but absent from all chunks is reported as a `COVERAGE WARNING` to `stderr`. This catches silent data loss.

Usage

Programmatic API

```
from src.ingestion.parser_chunker import process_and_chunk, save_chunks_to_json

chunks = process_and_chunk(pdf_path="data/my_manual.pdf")
save_chunks_to_json(chunks, output_path="data/processed/my_manual_chunks.json")
```

FastAPI Endpoint

```
curl -X POST http://localhost:8000/parse-manual \
  -F "file=@data/my_manual.pdf"

# Returns: JSON array of chunks with all metadata fields.
```

Extension for New Documents

When integrating a new manual:

1. **Inspect the PDF** to identify error code format, sub-heading labels, and section structure.
2. **Add patterns** to `_ERROR_CODE_PATTERNS`, `_SUB_HEADING_RE`, and `_SECTION_HEADING_RE` if needed.
3. **Update the test document table** at the top of this doc.
4. **Run coverage validation** and check `stderr` for any coverage warnings.
5. **Spot-check** a few chunks in the output JSON to ensure boundaries are correct.

See `src/ingestion/parser_chunker.py` source for exact pattern definitions.